

TUTORIAL DE AULA

Integração de Dados: consumindo uma API externa e gravando o resultado em banco de dados

Passo a passo para criar um banco PostgreSQL na nuvem, enriquecer registros de clientes com dados de endereço via API ViaCEP e gravar o resultado com Apache Hop.

Disciplina	Integração de Dados
Professor	Max Callil
Data da aula	14/09/2026
Ferramentas	Neon (PostgreSQL), DBeaver, Apache Hop, API ViaCEP

Objetivo

Orientar, passo a passo, como provisionar um banco de dados PostgreSQL na nuvem, cadastrar uma base de clientes, e usar o Apache Hop para consultar uma API pública de CEP e gravar os endereços retornados em uma tabela relacional — reproduzindo, de forma organizada, o fluxo completo demonstrado em aula.

1 Criar um projeto no Neon (banco PostgreSQL na nuvem)

Crie uma conta na plataforma Neon e inicie um novo projeto de banco de dados PostgreSQL. Defina um nome para o projeto e selecione a região mais próxima (por exemplo, AWS South America East 1 – São Paulo).

Create project

Project name: Postgres version:

Region:

Select the region closest to your application.

Enable Neon Auth: ☒ Neon Auth adds ready-to-use authentication to your app — users and sessions are stored directly in your database. You can also enable it later in project settings.

Region
AWS South America East 1 (São Paulo)

Your free plan includes:

- 0.5 GB storage
- Scales to zero when inactive
- Autoscaling up to 2 CU
- 10 branches per project

2 Obter a string de conexão

Com o projeto criado, acesse a tela de conexão do banco. Ali ficam disponíveis o branch, o banco de dados, o usuário (role) e a connection string, que serão usados para conectar outras ferramentas ao banco.

Connect to your database

Branch

production Default

Compute

Primary Active

Database

neondb

Role

neondb_owner

[Reset password](#)

Connection string

☒ Connection pooling

```
postgresql://neondb_owner:*****@ep-calm-fog-acbpefj0-pooler.sa-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
```

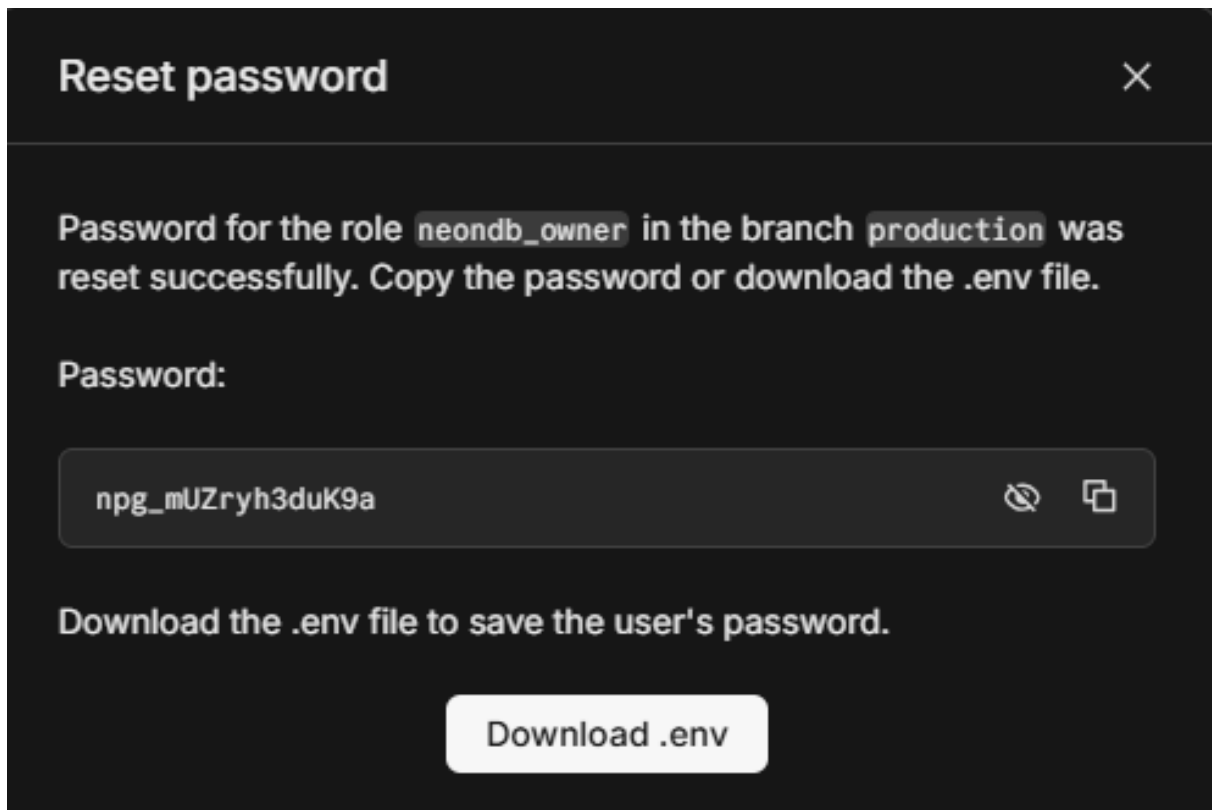
Copy snippet

Show password

Your password is saved in a secure storage vault. [Learn more about connecting](#)

3 Resetar a senha e guardar as credenciais

Gere uma nova senha para o usuário do banco (role) e guarde-a com segurança — ela será necessária para configurar a conexão em outras ferramentas. Também é possível baixar um arquivo `.env` com as credenciais completas. Guarde especialmente o host (trecho entre o "@" e ".tech" da connection string).



Observação: As credenciais utilizadas neste tutorial (usuário, senha e host) pertencem a um projeto de teste criado apenas para fins didáticos e devem ser substituídas pelas credenciais do seu próprio projeto.

4 Criar as tabelas no banco de dados

O banco de dados foi criado localmente com o auxílio do DBeaver: instale o DBeaver na sua máquina e crie uma nova conexão PostgreSQL, informando os dados obtidos no passo anterior (host, banco, usuário e senha do Neon). Com a conexão estabelecida, crie as três tabelas que serão utilizadas no exercício:

```
CREATE TABLE cliente (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100),  
    cep VARCHAR(8)  
);  
  
CREATE TABLE endereco (  
    id SERIAL PRIMARY KEY,  
    id_cliente INTEGER,  
    cep VARCHAR(8),  
    logradouro VARCHAR(200),  
    bairro VARCHAR(100),  
    cidade VARCHAR(100),  
    uf CHAR(2)  
);  
  
CREATE TABLE log_erros (  
    id SERIAL PRIMARY KEY,  
    cep VARCHAR(8),  
    motivo VARCHAR(200),  
    data_erro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

A tabela cliente armazena os dados de origem; endereco receberá o resultado da consulta à API; e log_erros fica disponível para registrar eventuais falhas de integração.

5 Popular a tabela de clientes

Insira alguns registros de teste na tabela cliente, incluindo propositalmente um CEP inválido para validar o tratamento de erros mais adiante:

```
INSERT INTO cliente (nome, cep)
VALUES
  ('João', '01001000'),
  ('Maria', '13010000'),
  ('Pedro', '20040002'),
  ('Ana', '13870000'),
  ('Cliente Inválido', '99999999');
```

Confira se os dados foram inseridos corretamente:

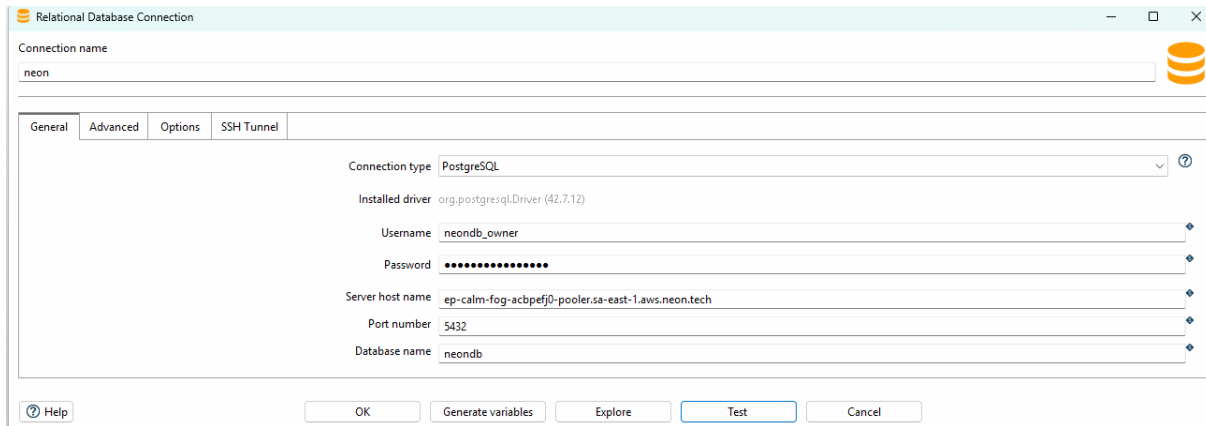
```
SELECT *
FROM cliente;
```

6 Instalar e abrir o Apache Hop

Baixe o Apache Hop, extraia o arquivo .zip e execute o arquivo hop-gui para abrir a aplicação. O Hop será usado para orquestrar a consulta ao banco, a chamada à API e a gravação do resultado.

7 Criar a conexão com o banco de dados no Hop

Dentro do Apache Hop, crie uma nova pipeline e configure uma nova conexão relacional do tipo PostgreSQL, informando o usuário, a senha, o host, a porta (5432) e o nome do banco de dados (neondb) obtidos anteriormente no Neon.



The screenshot shows the 'Relational Database Connection' dialog box in Apache Hop. The 'Connection name' field is set to 'neon'. The 'General' tab is selected, showing the following configuration:

- Connection type: PostgreSQL
- Installed driver: org.postgresql.Driver (42.7.12)
- Username: neondb_owner
- Password: [Redacted]
- Server host name: ep-calm-fog-acbpefj0-pooler.sa-east-1.aws.neon.tech
- Port number: 5432
- Database name: neondb

At the bottom, there are buttons for 'Help', 'OK', 'Generate variables', 'Explore', 'Test', and 'Cancel'.

8 Criar o transform "Table input"

Adicione um transform "Table input" ao canvas e configure a conexão criada no passo anterior. Escreva a consulta SQL que monta, para cada cliente, a URL de consulta da API ViaCEP a partir do CEP cadastrado:

```
SELECT
  id,
  nome,
  cep,
  'https://viacep.com.br/ws/' || cep || '/json/' AS url
FROM cliente
```

Table input

Transform name Table input

Connection neon

Load SQL from file Browse...

SQL Get SQL select statement...

```
SELECT
  id,
  nome,
  cep,
  'https://viacep.com.br/ws/' || cep || '/json/' AS url
FROM cliente
```

Line 6 Column 13

Replace variables in script ☐

Insert data from transform

Execute for each row ☐

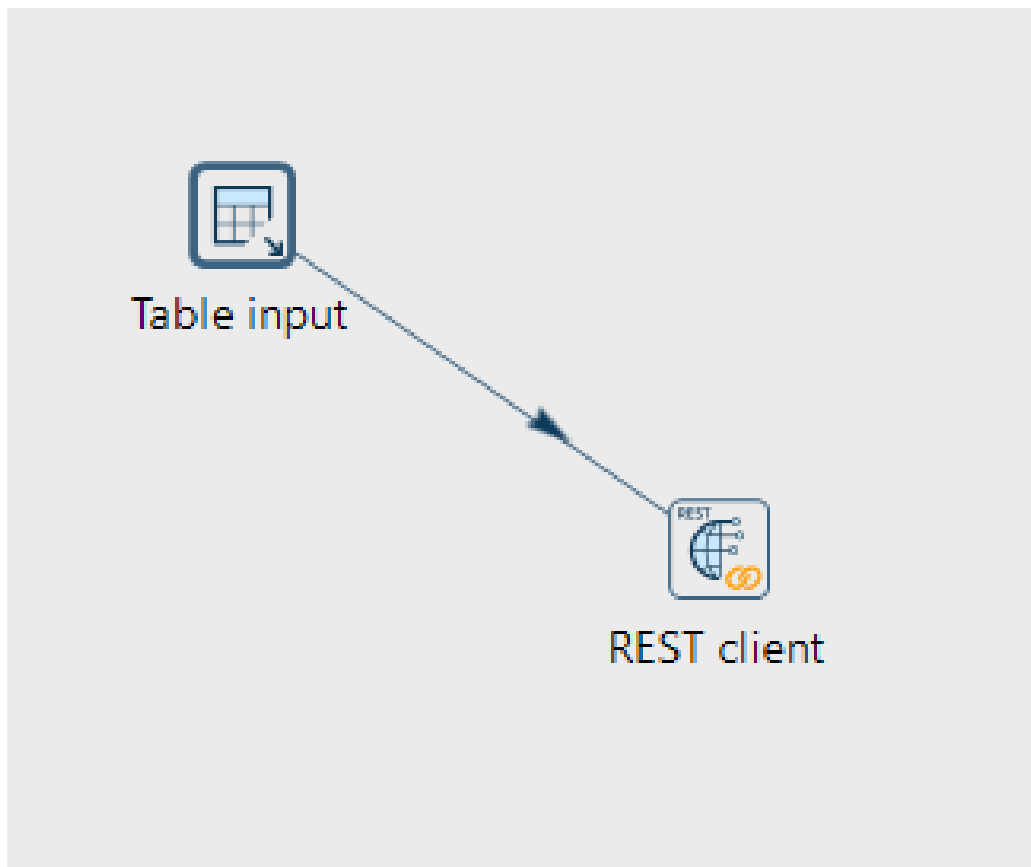
Limit size 0

Help OK Preview Cancel

Use o botão "Preview" para conferir se a consulta retorna os dados e as URLs esperadas.

9 Conectar o "Table input" ao "REST client"

Adicione um transform "REST client" ao canvas e crie um hop (uma seta de ligação) partindo do "Table input" até ele, para que os dados de cada cliente sigam para a chamada da API.



10 Configurar o "REST client" para consumir a API ViaCEP

Configure o transform "REST client" para aceitar a URL a partir do campo url gerado no passo anterior, com método HTTP GET. Defina o nome do campo de saída (por exemplo, response) que receberá o corpo da resposta da API.

The screenshot shows the "REST client" configuration window. The "Transform name" is set to "REST client". The "General" tab is selected, showing the following settings:

- REST Connection:** A dropdown menu with a search icon.
- URL:** `https://viacep.com.br/ws/`
- Accept URL from field:** ☒ (checked)
- URL field name:** `url`
- HTTP method:** `GET`
- Get Method from field:** ☐ (unchecked)
- Method field name:** (empty)
- Body field:** (empty)
- Application type:** `TEXT PLAIN`
- Connection timeout (milliseconds):** `10000`
- Read timeout (milliseconds):** `10000`

The "Output fields" section shows:

- Result field name:** `response`
- Result is binary:** ☐ (unchecked)
- HTTP status code field name:** `status`
- Response time (milliseconds) field name:** (empty)
- Response header field name:** (empty)

At the bottom, there are "Help", "OK", and "Cancel" buttons.

11 Criar o transform "JSON input"

Adicione um "JSON input" e conecte-o à saída do "REST client". Na aba "File", marque a opção que indica que a origem dos dados vem de um transform anterior e selecione o campo response como fonte do conteúdo JSON.

JSON input

Transform name: JSON input

File | Content | Fields | Additional output fields

Source from field

Source is from a previous transform: ☒

Select field: response

Use field as file names: ☐

Read source as URL: ☐

Do not pass field downstream: ☐

File or directory:

Regular Expression:

Exclude Regular Expression:

Selected files:

#	File/Directory	Wildcard (RegExp)	Exclude wildcard	Required	Include
1					

Show filename(s)...

Help OK Preview Cancel

12 Mapear os campos do JSON retornado pela API

Na aba "Fields", informe o caminho de cada campo do JSON que deve ser extraído da resposta da API: cep, logradouro, bairro, localidade e uf. Não é necessário preencher a coluna "Type".

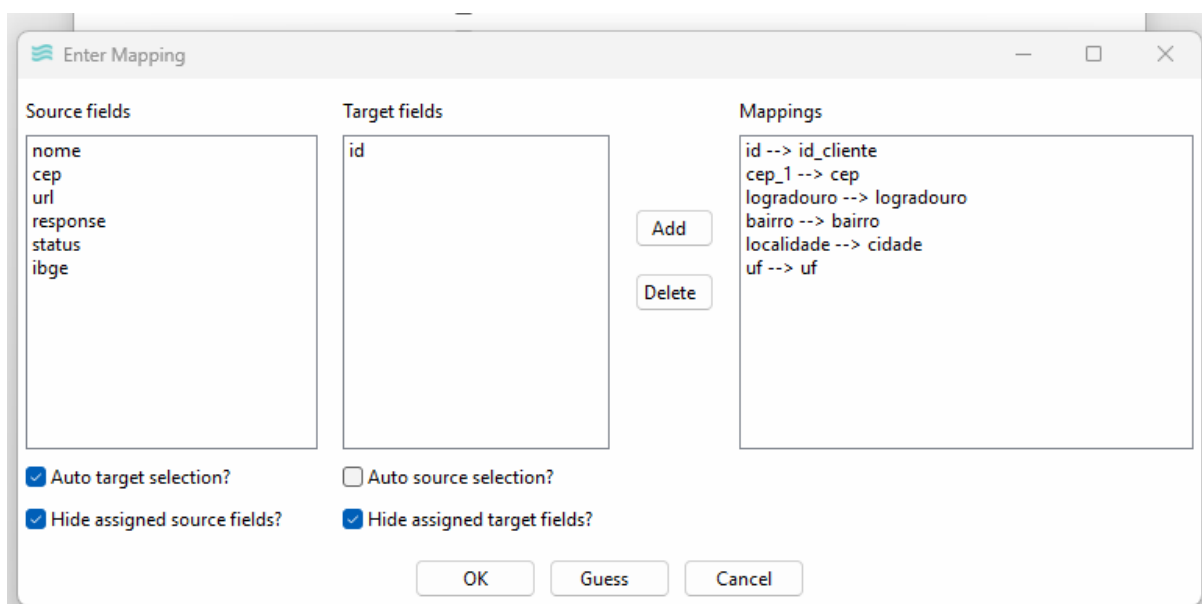
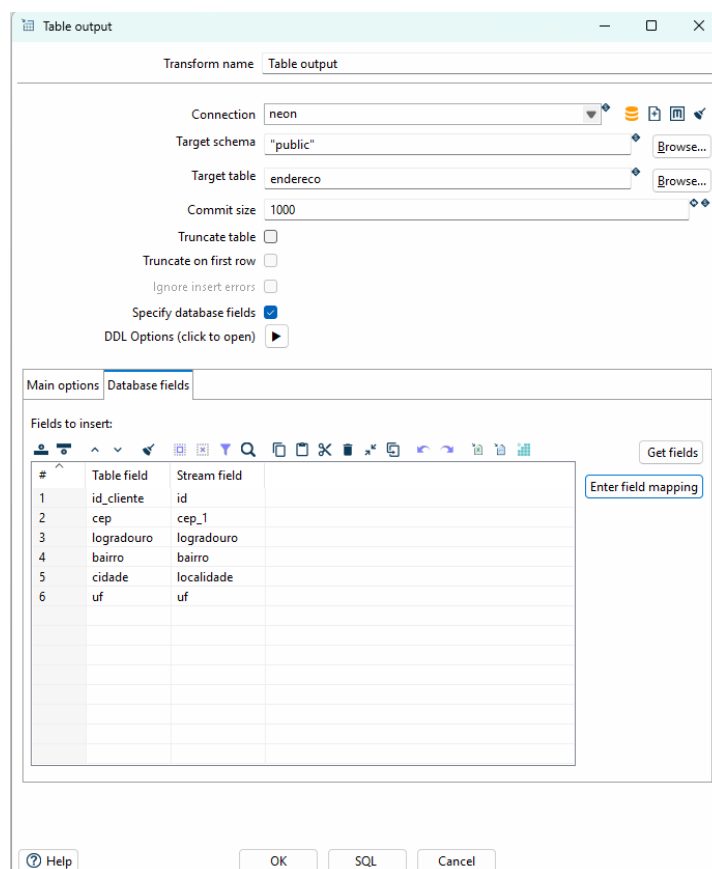
Transform name: JSON input

#	Name	Path	Type	Format	Length	Precision	Currency	Decimal	Group	Trim type	Rej
1	cep_1	cep	None							none	N
2	logradouro	logradouro	None							none	N
3	bairro	bairro	None							none	N
4	localidade	localidade	None							none	N
5	uf	uf	None							none	N
6	ibge	ibge	None							none	N

Buttons: Select fields, Select fields from snippet, Help, OK, Preview, Cancel

13 Criar o transform "Table output" e mapear os campos

Adicione um "Table output" conectado à saída do "JSON input", selecione a conexão com o banco, o schema public e a tabela de destino endereco. Marque a opção "Specify database fields" e use "Enter field mapping" para associar cada campo do fluxo à coluna correspondente da tabela:



O mapeamento final deve relacionar: id → id_cliente, cep_1 → cep, logradouro → logradouro, bairro → bairro, localidade → cidade e uf → uf.

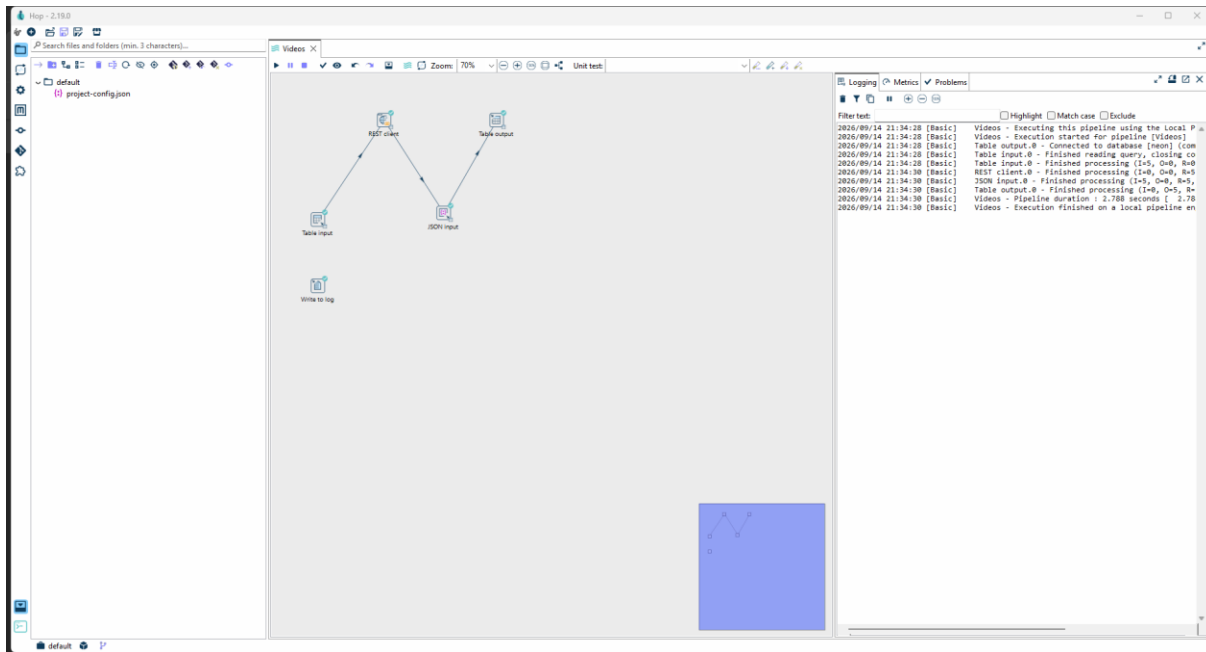
14 Ajustar o tamanho da coluna cep na tabela endereco

Como alguns CEPs retornados pela API podem incluir hífen, aumente o tamanho da coluna cep na tabela endereco antes de executar a carga completa:

```
ALTER TABLE endereco  
ALTER COLUMN cep TYPE varchar(9);
```


15 Executar o pipeline completo

Com os quatro transforms conectados (Table input → REST client → JSON input → Table output), execute a pipeline no Apache Hop e acompanhe o log de execução para confirmar que todos os registros foram processados sem erros.



16 Conferir o resultado no banco de dados

De volta ao DBeaver (ou outro cliente SQL), consulte a tabela `endereco` para verificar se os dados de logradouro, bairro, cidade e UF foram gravados corretamente para cada cliente:

```
SELECT *
FROM endereco;
```

	id	id_cliente	cep	logradouro	bairro	cidade	uf
1	1	1	01001-000	Praça da Sé	Sé	São Paulo	S
2	2	2	13010-000	Avenida Doutor Moraes Salles	Centro	Campinas	S
3	3	3	20040-002	Avenida Rio Branco	Centro	Rio de Janeiro	R
4	4	4	[NULL]	[NULL]	[NULL]	[NULL]	[I]
5	5	5	[NULL]	[NULL]	[NULL]	[NULL]	[I]

Registros cujo CEP era inválido (como o do "Cliente Inválido") devem aparecer com os campos de endereço em branco (NULL), já que a API não retornou dados para eles — esse é justamente o tipo de caso que a tabela `log_erros` poderia ser usada para registrar em um cenário de produção.

Ao final da execução, a tabela `endereco` estará populada com os dados retornados pela API ViaCEP para cada CEP cadastrado em cliente, concluindo o fluxo de integração de dados demonstrado em aula.